

软件学院综合性、设计性实验报告

专业:计算机科学与技术 班级:18 计科移动开发班 2020—2021 学年第一学期

课程名称	编译原理	指导教师	张聪品
学号姓名	1828324013 杨国旭		
实验地点	过街楼 404 实验室	实验时间	2020.10.01
项目名称	词法分析器的构造	实验类型	综合性/设计性

一、实验目的

设计并实现一个词法分析器，实现对源程序文本文件的读取，并能够对该源程序中的所有单词进行分类，指出其所属类型，实现简单的词法分析操作。

二、实验仪器或设备

实验环境:Microsoft Visual Studio 2012

编程环境:C++

三、总体设计（设计原理、设计方案及流程等）

待检测的源语言为类 C 语言(即缺少函数、语法识别的 C 代码)

(1) 能够将源语言输出

(2) 能够将该源程序中所有的单词根据其所属类型（整数、保留字、运算符、标识符等。定义类 C 语言中的标识符只能以字母或下划线开头）进行归类显示，例如：识别保留字：if、int、for、while、do、return、break、continue 等，其他的都识别为标识符；常数为无符号整形数；运算符包括：+、-、*、/、=、>、<、>=、<=、!=等；分隔符包括：,、;、{、}、(、)等。

四、实验步骤（包括主要步骤、代码分析等）

```
//main.cpp
```

```
#include<iostream>
```

```
#pragma warning (disable:4996)
```

```
using namespace std;
```

```
#define MAX 10
```

```
/*
```

```
保留字|关键字：1
```

```
操作符|运算符：2
```

```
分界符：3
```

```
标识符：4
```

```
常数：5
```

```
无识别：6
```

```
*/
```

```
char ch = ' ';
```

```
char*keyWord[10]={ "void","main","break","include","begin","end","if","else","while",  
"switch" };
```

```
char token[20];//定义获取的字符
```

```
//判断是否是关键字
```

```

bool isKey(char * token)
{
    for (int i = 0; i < MAX; i++)
    {
        if (strcmp(token, keyWord[i]) == 0)
            return true;
    }
    return false;
}
//判断是否是字母
bool isLetter(char letter)
{
    if ((letter >= 'a' && letter <= 'z') || (letter >= 'A' && letter <= 'Z'))
        return true;
    else
        return false;
}
//判断是否是数字
bool isDigit(char digit)
{
    if (digit >= '0' && digit <= '9')
        return true;
    else
        return false;
}
//词法分析
void analyze(FILE *fpin)
{
    while ((ch = fgetc(fpin)) != EOF) {
        if (ch == ' ' || ch == '\t' || ch == '\n') {}
        else if (isLetter(ch)) {
            char token[20] = { '\0' };
            int i = 0;
            while (isLetter(ch) || isDigit(ch)) {
                token[i] = ch;
                i++;
                ch = fgetc(fpin);
            }
            //回退一个指针
            fseek(fpin, -1, SEEK_CUR);
            if (isKey(token)) {
                //关键字
                cout << token << "\t1" << "\t 关键字" << endl;
            }
        }
    }
}

```

```

else {
    //标识符
    cout << token << "\t4" << "\t 标识符" << endl;
}
}
else if (isDigit(ch) || (ch == '.'))
{
    int i = 0;
    char token[20] = { '\0' };
    while (isDigit(ch) || (ch == '.' && isDigit(fgetc(fpin))))
    {
        if (ch == '.') fseek(fpin, -1, SEEK_CUR);
        token[i] = ch;
        i++;
        ch = fgetc(fpin);
    }
    fseek(fpin, -1, SEEK_CUR);
    //属于无符号常数
    cout << token << "\t5" << "\t 常数" << endl;
}
else switch (ch) {
    //运算符
case '+': {
    ch = fgetc(fpin);
    if (ch == '+') cout << "++" << "\t2" << "\t 运算符" << endl;
    if (ch == '=') cout << "+=" << "\t2" << "\t 运算符" << endl;
    else {
        cout << "+" << "\t2" << "\t 运算符" << endl;
        fseek(fpin, -1, SEEK_CUR);
    }
}break;
case '-': {
    ch = fgetc(fpin);
    if (ch == '-') cout << "--" << "\t2" << "\t 运算符" << endl;
    if (ch == '=') cout << "-=" << "\t2" << "\t 运算符" << endl;
    else {
        cout << "-" << "\t2" << "\t 运算符" << endl;
        fseek(fpin, -1, SEEK_CUR);
    }
}break;

case '*': {
    ch = fgetc(fpin);
    if (ch == '=') cout << "*=" << "\t2" << "\t 运算符" << endl;

```

```

else
{
    cout << ch << "\t2" << "\t 运算符" << endl;
    fseek(fpin, -1, SEEK_CUR);
}
} break;

case '/': {
    ch = fgetc(fpin);
    if (ch == '=') cout << "/" << "\t2" << "\t 运算符" << endl;
    else
    {
        cout << ch << "\t2" << "\t 运算符" << endl;
        fseek(fpin, -1, SEEK_CUR);
    }
} break;

//分界符
case '(': cout << ch << "\t3" << "\t 分界符" << endl; break;
case ')': cout << ch << "\t3" << "\t 分界符" << endl; break;
case '[': cout << ch << "\t3" << "\t 分界符" << endl; break;
case ']': cout << ch << "\t3" << "\t 分界符" << endl; break;
//case '!': cout << ch << "\t3" << "\t 分界符" << endl; break;
case '{': cout << ch << "\t3" << "\t 分界符" << endl; break;
case '}': cout << ch << "\t3" << "\t 分界符" << endl; break;

//运算符
case '=': {
    ch = fgetc(fpin);
    if (ch == '=') cout << "==" << "\t2" << "\t 运算符" << endl;
    else {
        cout << "=" << "\t2" << "\t 运算符" << endl;
        fseek(fpin, -1, SEEK_CUR);
    }
} break;

case ':': {
    ch = fgetc(fpin);
    if (ch == '=') cout << "==" << "\t2" << "\t 运算符" << endl;
    else {
        cout << ":" << "\t2" << "\t 运算符" << endl;
        fseek(fpin, -1, SEEK_CUR);
    }
} break;

case '>': {
    ch = fgetc(fpin);
    if (ch == '=') cout << ">=" << "\t2" << "\t 运算符" << endl;

```

```

        else {
            cout << ">" << "\t2" << "\t 运算符" << endl;
            fseek(fpin, -1, SEEK_CUR);
        }
    }break;
    case '<': {
        ch = fgetc(fpin);
        if (ch == '=')cout << "<=" << "\t2" << "\t 运算符" << endl;
        else {
            cout << "<" << "\t2" << "\t 运算符" << endl;
            fseek(fpin, -1, SEEK_CUR);
        }
    }break;
    //无识别
    default: cout << ch << "\t6" << "\t 无识别符" << endl;
    }
}
}
int main() {
    char input[30];
    FILE *fpin;
    cout << "请输入源文件名: \n" << endl;
    for (;;) {
        cin >> input;
        if ((fpin = fopen(input, "r")) != NULL)
            break;
        else
            cout << "路径输入错误" << endl;
    }
    cout << "*****词法分析结果*****" << endl;
    analyze(fpin);
    fclose(fpin);
}
//wrUtils.h

#pragma once
#include<iostream>
#include<fstream>
#include<string>
#include<cassert>
#include<vector>
#include<stdio.h>
using namespace std;

```

```

char split(char *a, char b) {
    int tag;
    tag=strlen(a)+1;
    char* buff = new char[tag];
    for (int i = 0; i < strlen(a); i++) {
        buff[i] = a[i];
    }
    for (int i = 0; i < 1; i++) {
        buff[i + strlen(a)] = b;
    }
    return *buff;
}

```

```

vector<string>getKWList() {
    vector<string>keyWordList;
    //keyWordList = { "False", "None", "True", "and", "as", "assert", "break",
    "class", "continue", "def", "del", "elif", "else", "except", "finally", "for", "from",
    "global", "if", "import", "in", "is", "lambda", "nonlocal", "not", "or", "pass", "raise",
    "return", "try", "while", "with", "yield" };
    return keyWordList;
}

```

```

char *getSymbolList() {
    char symbolList[] =
    { 32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,58,59,60,61,62,63,64,91,92,93,94,95,96,
    123,124,125,126 };
    return symbolList;
}

```

```

int getWordCount(string file) {
    ifstream ifs;
    char ch;
    ifs.open(file);
    int tag = 0;
    while (ifs.get(ch)) {
        tag++;
    }
    ifs.close();
    return tag;
}

```

```

vector<string>getPyKwlist() {
    string filename = "pyKWlist.txt";
    vector<string>keyWordList;
    int tag = getWordCount(filename);
    char *p,ch;
    p = new char[tag];
    ifstream file;
    file.open(filename);
    int a = 0;
    while (file.get(ch) && a < tag) {
        p[a]= ch;
        a++;
    }/*
    char reg = ',';
    char *all = new char[tag];
    for (int i = 0; i < tag; i++) {
        if (p[i]!=reg) {
            *all=split(all,p[i]);
        }
    }
    for (int i = 0; i < strlen(all);i++){
        cout << all[i];
    }
    */
    //keyWordList = { "False", "None", "True", "and", "as", "assert", "break",
    "class", "continue", "def", "del", "elif", "else", "except", "finally", "for", "from",
    "global", "if", "import", "in", "is", "lambda", "nonlocal", "not", "or", "pass", "raise",
    "return", "try", "while", "with", "yield" };
    return keyWordList;
}

void write(string file, string data) {
    ofstream ofs;
    ofs.open(file, ios::out);
    ofs << data << endl;
}

void readLine(string file) {
    ifstream infile;
    infile.open(file.data());    //将文件流对象与文件连接起来
    assert(infile.is_open());    //若失败,则输出错误消息,并终止程序运行

    string s;
    while (getline(infile, s))

```

```

    {
        cout << s << endl;
    }
    infile.close();          //关闭文件输入流
}

void readWithSkip(string file)
{
    ifstream infile;
    infile.open(file.data()); //将文件流对象与文件连接起来
    assert(infile.is_open()); //若失败,则输出错误消息,并终止程序运行

    char c;
    infile >> noskipws;
    while (!infile.eof()){
        infile >> c;
        cout << c << endl;
    }
    infile.close();          //关闭文件输入流
}

void readWithoutSkip(string file)
{
    ifstream infile;
    infile.open(file.data()); //将文件流对象与文件连接起来
    assert(infile.is_open()); //若失败,则输出错误消息,并终止程序运行

    char c;
    while (!infile.eof())
    {
        infile >> c;
        cout << c << endl;
    }
    infile.close();          //关闭文件输入流
}

```

//待识别的源语言代码

//data.c

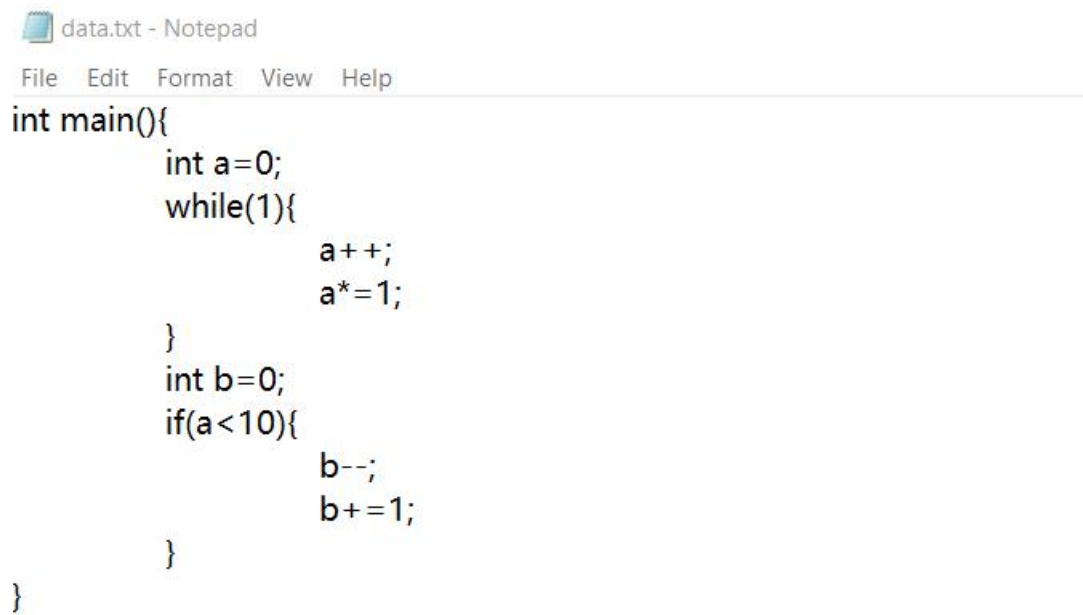
```

int main(){
    int a=0;
    while(1){

```



```
        a++;
        a*=1;
    }
    int b=0;
    if(a<10){
        b--;
        b+=1;
    }
}
```



A screenshot of a Notepad window titled "data.txt - Notepad". The window has a menu bar with "File", "Edit", "Format", "View", and "Help". The code inside is as follows:

```
int main(){
    int a=0;
    while(1){
        a++;
        a*=1;
    }
    int b=0;
    if(a<10){
        b--;
        b+=1;
    }
}
```

五、实验结果

C:\WINDOWS\system32\cmd.exe

请输入源文件名:

data.txt

*****待检测的源语言代码*****

```
int main() {
    int a=0;
    while(1) {
        a++;
        a*=1;
    }
    int b=0;
    if(a<10) {
        b--;
        b+=1;
    }
}
```

*****词法分析结果*****

int	4	标识符
main	1	关键字
(	3	分界符
)	3	分界符
{	3	分界符
int	4	标识符
a	4	标识符
=	2	运算符
0	5	常数
;	6	无识别符
while	1	关键字
(	3	分界符
1	5	常数
)	3	分界符

```
C:\WINDOWS\system32\cmd.exe

;          6          无识别符
a          4          标识符
*=         2          运算符
1          5          常数
;          6          无识别符
}          3          分界符
int        4          标识符
b          4          标识符
=          2          运算符
0          5          常数
;          6          无识别符
if         1          关键字
(          3          分界符
a          4          标识符
<          2          运算符
10         5          常数
)          3          分界符
{          3          分界符
b          4          标识符
--         2          运算符
-          2          运算符
-          2          运算符
;          6          无识别符
b          4          标识符
+=         2          运算符
1          5          常数
;          6          无识别符
}          3          分界符
}          3          分界符
Press any key to continue . . .
```

六、总结

通过这次实验清晰的理解了词法分析器的原理,对现代编译原理技术有了清晰的了解,为学习语法分析打好了基础

教师签名:

年 月 日